

Unit Testing Using Python

1. Validate Basic Calculator application through unittest module. use atleast 6 assertions methods in your testfile. //Attach all code snippets and outputs here.

```
1
2 def add(a,b):
3     return a+b
4
5 def sub(a,b):
6     return a-b
7
8 def multiply(a,b):
9     return a*b
10
11 def div(a,b):
12     if b == 0:
13         raise ValueError("cannot divide by zero!!!")
14     return a/b
15
16 x = input("Enter the first number")
17 y = input("Enter the second number")
18 z = input("choose operations")
19 if z == 1:
20     print("the answer is",add(x,y))
21 elif z == 2:
22     print("the answer is",sub(x,y))
23 elif z == 3:
24     print("the answer is",multiply(x,y))
25 elif z == 4:
26     print("the answer is",div(x,y))
27
```

```
1 import unittest
2 import basic_cal
3 class test1(unittest.TestCase):
4     def test_add(self):
5         self.assertEqual(basic_cal.add(4,3),7)
6         self.assertEqual(basic_cal.add(-1,-1),-2,"pass")
7         self.assertNotIsInstance(basic_cal.add(2.5,5),int)
8         self.assertGreaterEqual(basic_cal.add(2,2),4)
9     def test_sub(self):
10         self.assertEqual(basic_cal.sub(5,4),1)
11         self.assertEqual(basic_cal.sub(-5,6),-11)
12         self.assertIsInstance(basic_cal.sub(5.5,4),float)
13     def test_multiply(self):
14         self.assertEqual(basic_cal.multiply(4,3),12)
15         self.assertTrue(basic_cal.multiply(3,4)>0)
16         self.assertFalse(basic_cal.multiply(3,4)==10)
17     def test_div(self):
18         self.assertEqual(basic_cal.div(20,5),4)
19         self.assertRaises(ValueError,basic_cal.div,2,0)
20
21 if __name__ == "__main__":
22     unittest.main()
```

```

OK
PS C:\Users\Laptop> & C:/Users/Laptop/AppData/Local/Programs/Python/Python311/python.exe c:/Users/Laptop/OneDrive/Desktop/basic_calculator/test.py
....
-----
Ran 4 tests in 0.002s

OK
PS C:\Users\Laptop>

```

2. Validate any 2 module of your previous python application through unittest //. //Attach all code snippets and outputs here.

Addstudent feature

code:

```

# Addstudents button
def addstudents():
    global con,cur
    connectdb()
    q="SELECT * FROM login WHERE userid = '%i'"

    if e1.get() == '' or e2.get() == '' or e3.get() == '' or e4.get() == '' or e5.get() == '':
        messagebox.showerror("Student", "Fill All Details")
        return None

    cur.execute(q%(int(e2.get())))
    student = cur.fetchone()

    if student is None:
        qq='INSERT INTO Login VALUE("%s", "%i", "%i", "%s", "%i")'
        try:
            pas = int(e3.get())
        except ValueError:
            messagebox.showerror("Student", "Password Must be in Numbers")
            return None

        cur.execute(qq%(e1.get(),int(e2.get()),int(e3.get()),e4.get(),int(e5.get())))
        con.commit()
        win.destroy()
        messagebox.showinfo("Student", "Student Added")
        closedb()
        admin()
    else:
        messagebox.showerror("Student", "ID Already Taken.")
        closedb()

```

test script:

test_addstudents.py

```
1 import unittest
2 from unittest.mock import patch, MagicMock
3 import librarymanagement
4
5 class TestAddStudentsFunction(unittest.TestCase):
6     @patch('librarymanagement.admin')
7     @patch('librarymanagement.closedb')
8     @patch('librarymanagement.con')
9     @patch('librarymanagement.cur')
10    @patch('librarymanagement.connectdb')
11    @patch('tkinter.messagebox.showinfo')
12    @patch('tkinter.messagebox.showerror')
13    def test_successful_add(self, mock_error, mock_info, mock_connectdb, mock_cur, mock_con, mock_closedb, mock_admin):
14        # Test case for adding a student successfully
15        librarymanagement.e1 = MagicMock(); librarymanagement.e1.get.return_value = "Ali"
16        librarymanagement.e2 = MagicMock(); librarymanagement.e2.get.return_value = "1001"
17        librarymanagement.e3 = MagicMock(); librarymanagement.e3.get.return_value = "1234"
18        librarymanagement.e4 = MagicMock(); librarymanagement.e4.get.return_value = "CS"
19        librarymanagement.e5 = MagicMock(); librarymanagement.e5.get.return_value = "3333333333"
20        librarymanagement.win = MagicMock()
21        # No student with this ID exists
22        mock_cur.fetchone.return_value = None
23        librarymanagement.addstudents()
24        mock_cur.execute.assert_any_call('INSERT INTO Login VALUE("Ali","1001","1234","CS","3333333333")')
25        mock_info.assert_called_with("Student", "Student Added")
26        librarymanagement.win.destroy.assert_called_once()
27        mock_admin.assert_called_once()
28        mock_con.commit.assert_called_once()
29        @patch('librarymanagement.con')
30        @patch('librarymanagement.cur')
31        @patch('tkinter.messagebox.showerror')
32        @patch('librarymanagement.connectdb')
33        def test_student_id_exists(self, mock_connectdb, mock_error, mock_cur, mock_con):
34            librarymanagement.e1 = MagicMock(); librarymanagement.e1.get.return_value = "Zara"
35            librarymanagement.e2 = MagicMock(); librarymanagement.e2.get.return_value = "1002"
36            librarymanagement.e3 = MagicMock(); librarymanagement.e3.get.return_value = "1234"
37            librarymanagement.e4 = MagicMock(); librarymanagement.e4.get.return_value = "EE"
38
39            librarymanagement.e4 = MagicMock(); librarymanagement.e4.get.return_value = "EE"
40            librarymanagement.e5 = MagicMock(); librarymanagement.e5.get.return_value = "4444444444"
41            # Simulate student already exists
42            mock_cur.fetchone.return_value = ("Zara",)
43            librarymanagement.addstudents()
44            mock_error.assert_called_with("Student", "ID Already Taken.")
45        @patch('tkinter.messagebox.showerror')
46        @patch('librarymanagement.connectdb')
47        def test_empty_fields(self, mock_connectdb, mock_error):
48            # Test case for when fields are empty # Setup empty field
49            librarymanagement.e1 = MagicMock(); librarymanagement.e1.get.return_value = ""
50            librarymanagement.e2 = MagicMock(); librarymanagement.e2.get.return_value = "1003"
51            librarymanagement.e3 = MagicMock(); librarymanagement.e3.get.return_value = "1234"
52            librarymanagement.e4 = MagicMock(); librarymanagement.e4.get.return_value = "ME"
53            librarymanagement.e5 = MagicMock(); librarymanagement.e5.get.return_value = "9999999999"
54            librarymanagement.addstudents()
55            # Verify error message for empty field
56            mock_error.assert_called_with("Student", "Fill All Details")
57        @patch('tkinter.messagebox.showerror')
58        @patch('librarymanagement.cur')
59        @patch('librarymanagement.connectdb')
60        def test_invalid_password(self, mock_connectdb, mock_cur, mock_error):
61            # Test case for when password is invalid (non-numeric)
62            # Setup non-numeric password
63            librarymanagement.e1 = MagicMock(); librarymanagement.e1.get.return_value = "Sara"
64            librarymanagement.e2 = MagicMock(); librarymanagement.e2.get.return_value = "1004"
65            librarymanagement.e3 = MagicMock(); librarymanagement.e3.get.return_value = "abc"
66            librarymanagement.e4 = MagicMock(); librarymanagement.e4.get.return_value = "IT"
67            librarymanagement.e5 = MagicMock(); librarymanagement.e5.get.return_value = "7777777777"
68            mock_cur.fetchone.return_value = None
69            librarymanagement.addstudents()
70            # Verify error message for invalid password
```

```

57 |         librarymanagement.addstudents()
58 |         # Verify error message for invalid password
59 |         mock_error.assert_called_with("Student", "Password Must be in Numbers")
60 |
61 | if __name__ == '__main__':
62 |     unittest.main()
63 |

```

output:

```
PS C:\Users\khali\Library-Management> python -m unittest discover -s . -p "test_*.py" -v
```

```
>>> test_empty_fields (test_addstudents.TestAddStudentsFunction.test_empty_fields) ... ok
test_invalid_password (test_addstudents.TestAddStudentsFunction.test_invalid_password) ... ok
test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_test_student_id_exists) ... ok
on.test_successful_add) ...
test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_student_id_exists) ... ok
test_successful_add (test_addstudents.TestAddStudentsFunction.test_student_id_exists) ... ok
test_successful_add (test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_student_id_exists) ... ok
test_successful_add (test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_student_id_exists) ... ok
unction.test_student_id_exists) ... ok
test_successful_add (test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_student_id_exists) ... ok
test_successful_add (test_addstudents.TestAddStudentsFunction.test_successful_add) ... ok
```

```
Ran 4 tests in 0.032s
```

Delete Student feature

code:

```

# Deletestudents query
def deletestudents():
    connectdb()
    if e2.get()=='admin':
        q="SELECT * FROM login WHERE userid = '%i'"

        if e1.get() == '':
            messagebox.showerror("Delete", "Enter Correct ID")
            return None

        cur.execute(q%(int(e1.get())))
        book = cur.fetchone()

        if book is None:
            messagebox.showerror("Delete", "Student Not Found.")
            closedb()
        else:
            qq="DELETE FROM login WHERE userid = '%i'"
            cur.execute(qq%(int(e1.get())))
            con.commit()
            win.destroy()
            messagebox.showinfo("Delete", "Student Deleted Successfully.")
            closedb()
            admin()

    else:
        messagebox.showerror("Delete", "Password is Wrong.")
        closedb()

```

test script:

```
import unittest
from unittest.mock import patch, MagicMock
import librarymanagement

class TestDeleteStudentsFunction(unittest.TestCase):

    @patch('librarymanagement.admin')
    @patch('librarymanagement.closedb')
    @patch('librarymanagement.con')
    @patch('librarymanagement.cur')
    @patch('librarymanagement.connectdb')
    @patch('tkinter.messagebox.showinfo')
    @patch('tkinter.messagebox.showerror')
    def test_successful_delete(self, mock_error, mock_info, mock_connectdb, mock_cur, mock_con, m
        librarymanagement.e2 = MagicMock(); librarymanagement.e2.get.return_value = 'admin'
        librarymanagement.e1 = MagicMock(); librarymanagement.e1.get.return_value = "1001"
        librarymanagement.win = MagicMock()
        mock_cur.fetchone.return_value = ("Ali",)

        librarymanagement.deletestudents()

        mock_cur.execute.assert_any_call("DELETE FROM login WHERE userid = '%i'" % int(1001))
        mock_info.assert_called_with("Delete", "Student Deleted Successfully.")
        librarymanagement.win.destroy.assert_called_once()
        mock_con.commit.assert_called_once()
        mock_admin.assert_called_once()

    @patch('tkinter.messagebox.showerror')
    @patch('librarymanagement.connectdb')
    def test_empty_id_field(self, mock_connectdb, mock_error):
        librarymanagement.e2 = MagicMock(); librarymanagement.e2.get.return_value = 'admin'
        librarymanagement.e1 = MagicMock(); librarymanagement.e1.get.return_value = ""
        librarymanagement.deletestudents()
        mock_error.assert_called_with("Delete", "Enter Correct ID")

    @patch('tkinter.messagebox.showerror')
    @patch('librarymanagement.connectdb')
    def test_empty_id_field(self, mock_connectdb, mock_error):
        librarymanagement.e2 = MagicMock(); librarymanagement.e2.get.return_value = 'admin'
        librarymanagement.e1 = MagicMock(); librarymanagement.e1.get.return_value = ""
        librarymanagement.deletestudents()
        mock_error.assert_called_with("Delete", "Enter Correct ID")

    @patch('librarymanagement.closedb')
    @patch('tkinter.messagebox.showerror')
    @patch('librarymanagement.cur')
    @patch('librarymanagement.connectdb')
    def test_student_not_found(self, mock_connectdb, mock_cur, mock_error, mock_closedb):
        librarymanagement.e2 = MagicMock(); librarymanagement.e2.get.return_value = 'admin'
        librarymanagement.e1 = MagicMock(); librarymanagement.e1.get.return_value = "1234"
        mock_cur.fetchone.return_value = None
        librarymanagement.deletestudents()
        mock_error.assert_called_with("Delete", "Student Not Found.")

    @patch('tkinter.messagebox.showerror')
    def test_invalid_admin_password(self, mock_error):
        librarymanagement.e2 = MagicMock(); librarymanagement.e2.get.return_value = 'wrongpass'
        librarymanagement.e1 = MagicMock(); librarymanagement.e1.get.return_value = "1001"
        librarymanagement.deletestudents()
        mock_error.assert_called_with("Delete", "Password is Wrong.")

if __name__ == '__main__':
    unittest.main()
```

output:

▼ TERMINAL

```
PS C:\Users\khali\Library-Management> python -m unittest discover -s . -p "test_*.py" -v
>>
test_empty_fields (test_addstudents.TestAddStudentsFunction.test_empty_fields) ... ok
test_invalid_password (test_addstudents.TestAddStudentsFunction.test_invalid_password) ... ok
test_student_id_exists (test_addstudents.TestAddStudentsFunction.test_student_id_exists) ... ok
test_successful_add (test_addstudents.TestAddStudentsFunction.test_successful_add) ... ok
test_empty_id_field (test_deletestudents.TestDeleteStudentsFunction.test_empty_id_field) ... ok
test_invalid_admin_password (test_deletestudents.TestDeleteStudentsFunction.test_invalid_admin_password) ... ok
test_student_not_found (test_deletestudents.TestDeleteStudentsFunction.test_student_not_found) ... ok
test_successful_delete (test_deletestudents.TestDeleteStudentsFunction.test_successful_delete) ... ok

-----
Ran 8 tests in 0.069s
```

